



Python

DEVELOPPER AVEC UN LANGAGE PROCEDURAL



Objectifs

- Comprendre les bases de la programmation procédurale (variables, boucles, conditions, fonctions).
- Utiliser un environnement de développement pour écrire et déboguer du code.
- Compiler et exécuter un programme Maitriser les outils de gestion de version



Environnement.



IDE



Interpreter Python



Documentation

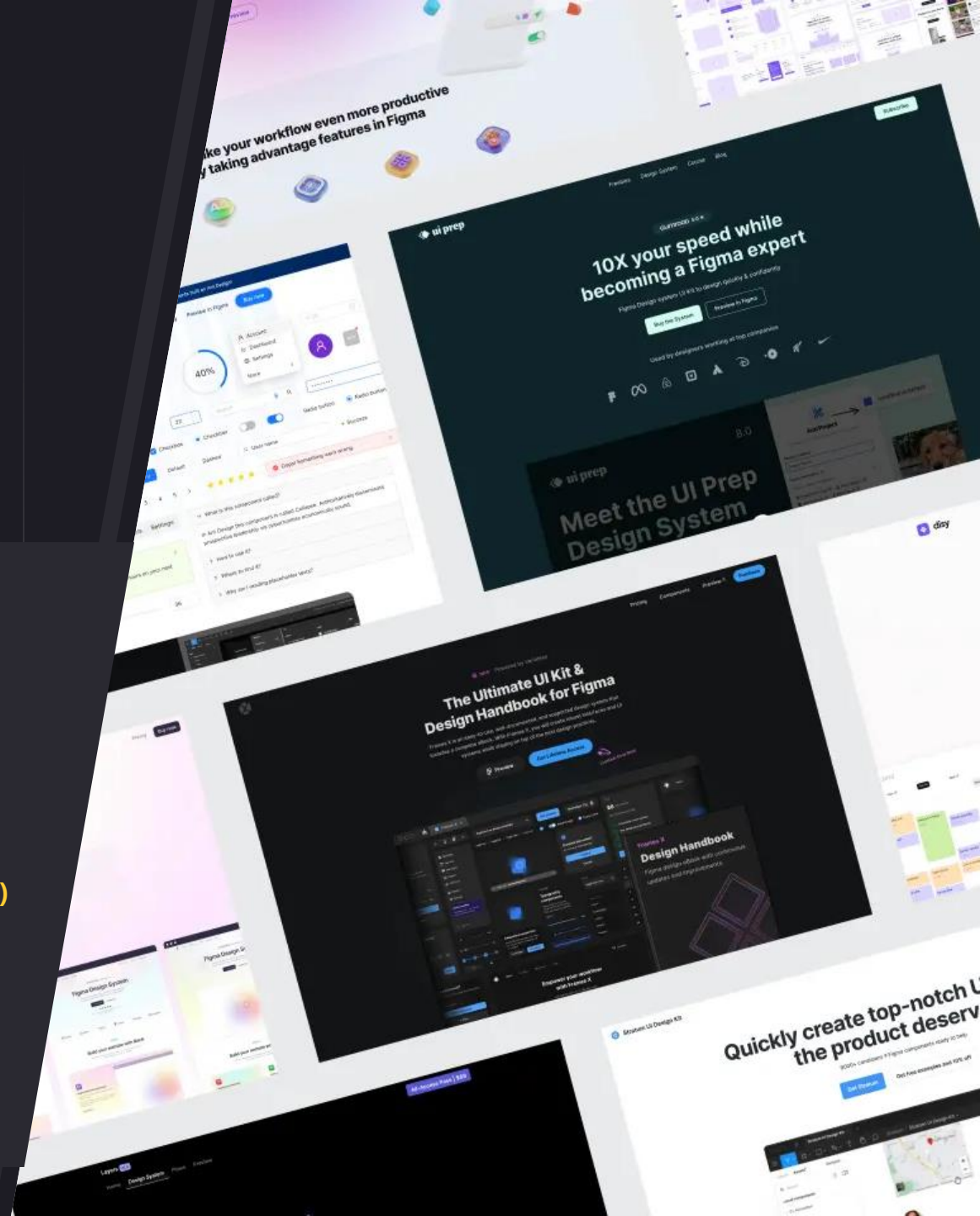
1. Python est un langage interprété

Inclus des gestionnaires de paquets (pour installer des bibliothèques)

Pas d'interpréteur Python = pas de Python exécuté.

C'est lui qui traduit code Python en instructions que l'ordinateur comprend.

Sur Windows cocher la case « PATH »



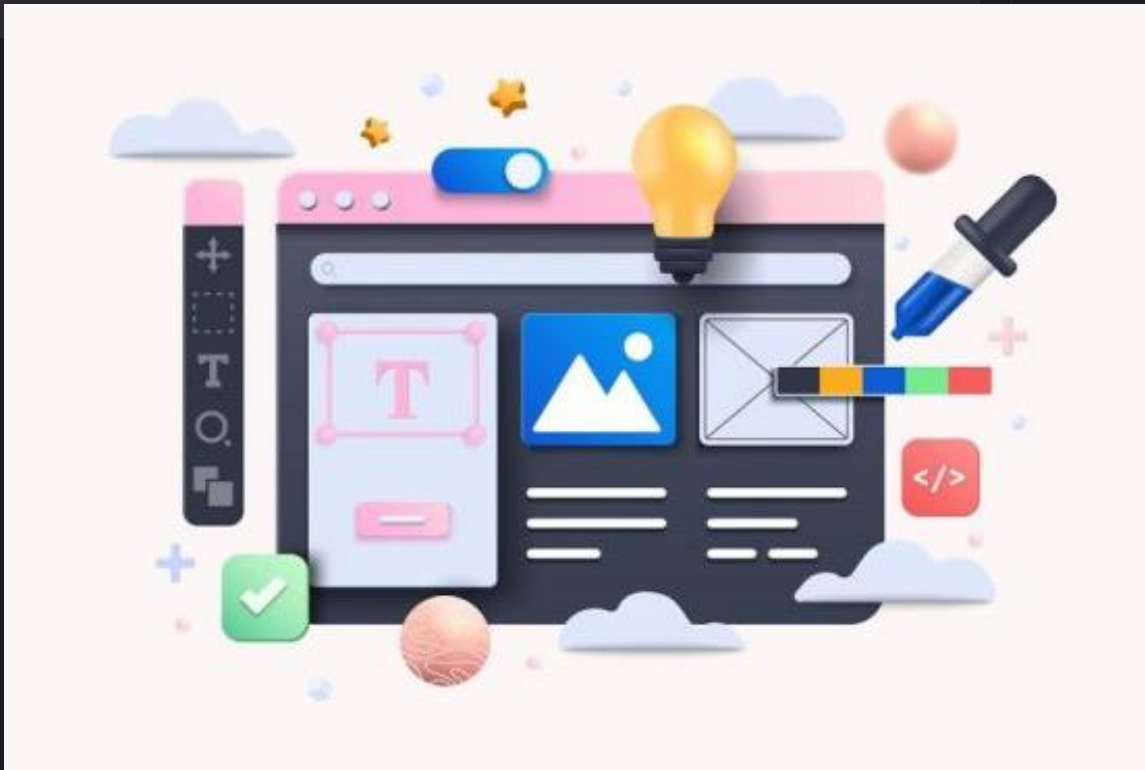


Généralités

Commentaires avec le #

Print() pour afficher dans la console

Nécessité de Run le code




```
1 # Variable = A container for a value
2 #           A variable behaves as if
3
4 # Strings
5 first_name = "Bro"
6 food = "pizza"
7 email = "Bro123@fake.com"
8
9 # Integers
10 age = 25
11 quantity = 3
12 num_of_students = 30
13
14 # Float
15 price = 10.99
16 gpa = 3.2
17 distance = 5.5
18
19 # Boolean
20 is_student = True
21 for_sale = False
```

Déclarations variables

En Python, pas besoin de déclarer le type : le langage est dynamique.

📌 Règles de nommage

- Commence par une lettre ou _
- Pas d'espace, pas de caractères spéciaux
- Sensible à la casse (Age ≠ age)

Il est possible de faire des affectations multiples

```
a, b, c = 1, 2, 3
x = y = 0 # Même valeur
```



```
main.py x
1 # Typecasting = the process of converting a variable from one data type to another
2 # str(), int(), float()
3
4 name = "Bro Code"
5 age = 25
6 gpa = 3.2
7 is_student = True
8
9 age = str(age)
10
11 age += "1"
12
13 print(age)
```

TypeCasting

Le **typecasting** permet de convertir une variable d'un type à un autre.

Pour vérifier le type d'une variable on utilisera **type(maVar)**

int()	int("10")	→ 10 (entier)
float	float("3.14")	→ 3.14 (flottant)
str()	str(25)	→ "25" (chaîne)
bool()	bool(0), bool("salut")	→ False, True



```
# input() = A function that prompts the user for input
# Returns the entered data as a string
```

```
name = input("What is your name?: ")
age = int(input("How old are you?: "))
```

```
age = age + 1
```

```
print(f"Hello {name}!")
print("HAPPY BIRTHDAY!")
print(f"You are {age} years old")
```

Input

Pour demander à l'utilisateur **de saisir des informations** depuis le clavier.

- **input()** renvoie toujours **une chaîne de caractères (str)**.
- Utiliser `int()` ou `float()` si besoin de conversion.

⚠ Attention

```
age = int(input("Ton âge ?"))
```

Si l'utilisateur tape du texte non numérique ❌ Erreur !



Exercice 1 : input & variables

```
/*
```

```
    1. Ecrire un programme qui demande la longueur puis la  
    largeur d'un rectangle et calcule l'aire de ce rectangle puis  
    l'affiche
```

```
*/
```

```
/*
```

```
    2. Ecrire un programme qui demande quel article acheter, son  
    prix, puis la quantité. On affiche ensuite une phrase  
    recapitulative du style « vous avez acheté 9 pizzas pour un  
    total de 999€ »
```

```
*/
```


Exercise 1 : corrections

```
1 # Exercise 1 Rectangle Area Calc
2
3 length = float(input("Enter the length: "))
4 width = float(input("Enter the width: "))
5 area = length * width
6
7 print(f"The area is: {area}cm2|")
```

```
# Exercise 2 Shopping Cart Program
```

```
item = input("What item would you like to buy?: ")
price = float(input("What is the price?: "))
quantity = int(input("How many would you like?: "))
total = price * quantity
```

```
print(f"You have bought {quantity} x {item}/s")
print(f"Your total is: ${total}")
```



```
# if = Do some code only IF some
# Else do something else

age = int(input("Enter your age

if age >= 100:
    print("You are too old to s:
elif age >= 18:
    print("You are now signed up
elif age < 0:
    print("You haven't been born
else:
    print("You must be 18+ to s:

if age >= 18
```

Condition **if / else**

Pour exécuter différents blocs de code selon des conditions logiques.

- **if** : teste une première condition.
- **elif** : (optionnel) teste d'autres conditions si la précédente est fausse
- **.else** : (optionnel) s'exécute si aucune condition n'est remplie.

⚠ Attention à l'indentation

- ✓ Indenter le code (généralement avec **4 espaces** ou **Tab**)



Exercice 2 : input & conditions

```
/*
```

```
    1. Ecrire un programme qui demande l'opérateur (« + - * / »)  
    2 chiffres, et opère le calcul en fonction de l'opérateur entré
```

```
*/
```

```
/*
```

```
    2. écrire un programme qui demande à l'utilisateur de saisir  
    une note sur 20. Le programme doit ensuite afficher la mention  
    associée ET indiquer si la note est valide.*/
```

Coorection exercices 2

```
# Demander la note à l'utilisateur
note = float(input("Entrez votre note sur 20 : "))

# Vérification de la validité de la note
if note < 0 or note > 20:
    print("Erreur : note invalide (doit être entre 0 et 20)")
else:
    # Détermination de la mention selon la note
    if note >= 16:
        print("Mention : Très bien")
    elif note >= 14:
        print("Mention : Bien")
    elif note >= 12:
        print("Mention : Assez bien")
    elif note >= 10:
        print("Mention : Passable")
    else:
        print("Mention : Insuffisant")
```


Python - Logical O

• not

x	not x
False	True
True	False

• and

x	y	x and y
False	False	False
False	True	False
True	False	False
True	True	True

• or

x	y	x or y
False	False	False
False	True	True
True	False	True
True	True	True

Opérateurs logiques

Les opérateurs logiques permettent de combiner ou inverser des conditions.

and

Vrai si les 2 conditions sont vraies

or

Vrai si au moins une condition est vraie

not

Inverse la valeur d'une condition

⚠ Eviter les if imbriqués



Utilise les opérateurs logiques pour simplifier les conditions longues au lieu de les imbriquer.




```
age = 13
temperature = 20
user_role = "admin"

# print("Positive" if num > 0 else "Nega
# result = "EVEN" if num % 2 == 0 else "
# max_num = a if a > b else b
# min_num = a if a < b else b
# status = "Adult" if age >= 18 else "Ch
# weather = "HOT" if temperature > 20 el
access_level = "Full Access" if user_rol

print(access_level)
```

Condition ternaire

Permet d'écrire une **condition simple en une seule ligne**.

résultat = valeur_si_vrai **if condition else** valeur_si_faux

C'est une alternative plus compacte à if/else. Pratique pour les affectations rapides ou les affichages simples.

⚠ Ne pas l'utiliser pour des conditions complexes



Methode **string**

Méthode	Description	Exemple
<code>len()</code>	Longueur de la chaîne	<code>len("Python")</code> → 6
<code>find(s)</code>	Position de la 1ère occurrence de <code>s</code>	<code>"bonjour".find("o")</code> → 1
<code>rfind(s)</code>	Position de la dernière occurrence de <code>s</code>	<code>"bonjour".rfind("o")</code> → 4
<code>capitalize()</code>	Met la 1re lettre en majuscule	<code>"python".capitalize()</code> → "Python"
<code>upper()</code>	Transforme en majuscules	<code>"hello".upper()</code> → "HELLO"
<code>lower()</code>	Transforme en minuscules	<code>"HELLO".lower()</code> → "hello"
<code>isdigit()</code>	Vérifie si la chaîne ne contient que des chiffres	<code>"123".isdigit()</code> → True
<code>isalpha()</code>	Vérifie si la chaîne ne contient que des lettres	<code>"abc".isalpha()</code> → True
<code>count(s)</code>	Compte le nombre d'occurrences de <code>s</code>	<code>"salut".count("s")</code> → 1
<code>replace(a, b)</code>	Remplace <code>a</code> par <code>b</code> dans la chaîne	<code>"pomme".replace("m", "n")</code> → "ponne"

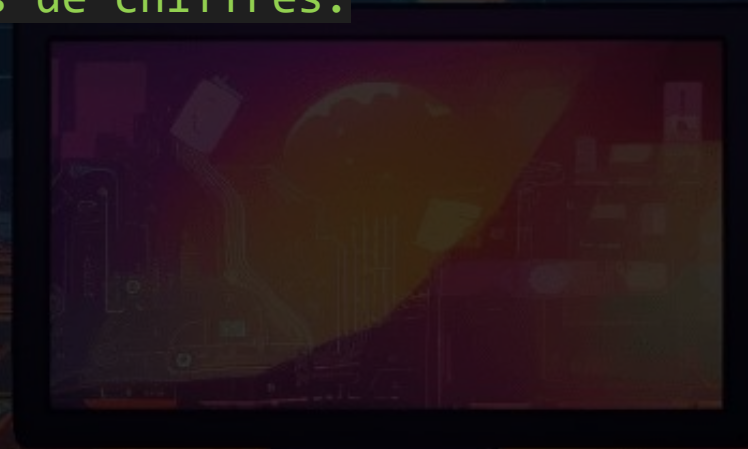


Exercice 3 : chaines et if

```
/*
```

```
    1. Ecrire un programme qui valide l'entrée d'un nom  
    utilisateur. Il doit contenir moins de 12 caractères, pas  
    d'espaces et pas de chiffres.
```

```
*/
```



Correction Exercice 3

```
username = input("Enter a username: ")

if len(username) > 12:
    print("Your username can't be more than 12 characters")
elif not username.find(" ") == -1:
    print("Your username can't contain spaces")
elif not username.isalpha():
    print("Your username can't contain numbers")
else:
    print(f>Welcome {username}")
```



```
# indexing = accessing elements of a  
# [start : end : step]  
  
credit_number = "1234-5678-9012-3456"  
  
last_digits = credit_number[-4:]  
print(f"XXXX-XXXX-XXXX-{last_digits}")
```

Syntaxe

Résultat

texte[0:4]	"Pyth"
texte[::2]	"Pto" (1 sur 2)
texte[1:]	"ython" (à partir de l'index 1)
texte[:3]	"Pyt" (jusqu'à l'index 2 inclus)
texte[::-1]	"nohtyP" (chaîne inversée)

Indexation String

maVar [début : fin : step]

Les index commencent à 0

✗ Une erreur se produit si l'index dépasse la taille de la chaîne

- texte[a:b] inclut l'indice a, mais exclut b
- L'indexation négative permet de partir de la fin
- Le step (pas) est optionnel et sert à sauter des caractères



```
py x
# indexing = accessing elements of a
# [start : end : step]

credit_number = "1234-5678-9012-3456"

last_digits = credit_number[-4:]
print(f"XXXX-XXXX-XXXX-{last_digits}")
```

Syntaxe	Résultat
<code>texte[0:4]</code>	<code>"Pyth"</code>
<code>texte[::2]</code>	<code>"Pto"</code> (1 sur 2)
<code>texte[1:]</code>	<code>"ython"</code> (à partir de l'index 1)
<code>texte[:3]</code>	<code>"Pyt"</code> (jusqu'à l'index 2 inclus)
<code>texte[::-1]</code>	<code>"nohtyP"</code> (chaîne inversée)

formatage (flags)

Pour afficher proprement du texte, des nombres ou des variables avec un format lisible et contrôlé.

Syntaxe	Description	Exemple de sortie
<code>f"{val:5}"</code>	Largeur fixe (aligné à droite)	<code>' 42'</code>
<code>f"{val:<5}"</code>	Aligné à gauche	<code>'42 '</code>
<code>f"{val:^5}"</code>	Centré	<code>' 42 '</code>
<code>f"{val:05}"</code>	Zéros en padding	<code>'00042'</code>
<code>f"{val:.2f}"</code>	2 décimales (nombre flottant)	<code>'3.14'</code>
<code>f"{pourcentage:.1%}"</code>	Format pourcentage	<code>0.25 → '25.0%'</code>

- Le formatage f-string est lisible, moderne, et très utilisé.
- Combine les {} avec des options de mise en forme : alignement, décimales, zéros...



Exercice 4 : while et formatage

```
/*
```

```
    1. Créer un petit jeu où l'utilisateur doit deviner un  
    nombre secret. Le programme utilise une boucle while pour répéter  
    les tentatives, et des f-strings avec flags pour formater les  
    messages.
```

```
*/
```

```
Tentative n°01 : Entrez un nombre : 5  
Trop bas !
```

```
Tentative n°02 : Entrez un nombre : 10  
Trop haut !
```

```
Tentative n°03 : Entrez un nombre : 7  
Bravo ! Vous avez trouvé en 03 tentatives.
```

Exercice 4 : correction

```
secret = 7
essai = 0
trouve = False

while not trouve:
    essai += 1
    guess = int(input(f"Tentative n°{essai:02} : Entrez un nombre : "))

    if guess < secret:
        print("Trop bas !")
    elif guess > secret:
        print("Trop haut !")
    else:
        print(f"Bravo ! Vous avez trouvé en {essai:02} tentatives.")
        trouve = True
```


Une boucle qui permet de parcourir une séquence (liste, chaîne, plage de nombres, etc.)

```
for variable in séquence:
    # bloc de code à répéter
```

```
for variable in séquence:
    # bloc de code à répéter
```

```
for variable in séquence:
    # bloc de code à répéter
```

À chaque tour, la variable prend la valeur suivante de la séquence.

range(x) génère les nombres de 0 à x-1

Syntaxe

```
range(n)
```

```
range(a, b)
```

```
range(a, b, step)
```

Résultat généré

de 0 à $n-1$

de a à $b-1$

de a à $b-1$, par palier de $step$



Exercice 5 : boucle for

```
/*  
    1. Demander deux nombres à l'utilisateur : un début et une  
    fin. Pour chaque entier entre ces deux bornes (incluses),  
    afficher : Le nombre S'il est pair ou impair */
```

Début : 3

Fin : 8

3 est impair

4 est pair

5 est impair

6 est pair

7 est impair

8 est pair

Exercice 5 : correction

```
# Demander les bornes à l'utilisateur
debut = int(input("Début : "))
fin = int(input("Fin : "))

# Parcourir la plage de nombres
for nombre in range(debut, fin + 1):
    if nombre % 2 == 0:
        print(f"{nombre} est pair")
    else:
        print(f"{nombre} est impair")
```

```
# -- Création d'un dictionnaire
```

```
utilisateur = {  
    "nom": "Alice",  
    "age": 25,  
    "ville": "Paris"  
}
```

```
# -- Accès aux valeurs
```

```
print(utilisateur["nom"])    # Alice  
print(utilisateur["age"])    # 25
```

Méthode	Description
<code>dict.keys()</code>	Liste des clés
<code>dict.values()</code>	Liste des valeurs
<code>dict.items()</code>	Liste (clé, valeur)
<code>dict.get("clé")</code>	Récupérer une valeur en sécurité
<code>dict.pop("clé")</code>	Supprimer une clé
<code>dict.clear()</code>	Vider le dictionnaire

dictionnaires (dict)

Un dictionnaire est une **collection de paires clé → valeur**.

C'est l'équivalent d'un objet en JavaScript.

```
# -- Création d'un dictionnaire
```

```
utilisateur = {  
    "nom": "Alice",  
    "age": 25,  
    "ville": "Paris"  
}
```

```
# -- Accès aux valeurs
```

```
print(utilisateur["nom"])    # Alice  
print(utilisateur["age"])    # 25
```

Ils disposent de méthodes utiles

Mutable (on peut le modifier).

Clés uniques et généralement de type str.

Très pratique pour représenter des objets, enregistrements ou configurations.



Exercice 6 : dictionnaires

Créer un petit carnet d'adresses en utilisant un dictionnaire pour stocker des informations sur des personnes.

Consignes : Crée un dictionnaire contact avec les clés suivantes : "nom" "telephone" "email"

Demande à l'utilisateur de saisir ces informations et remplis le dictionnaire.

Affiche le contact de façon lisible.

Bonus : Permettre à l'utilisateur de modifier le numéro de téléphone puis d'afficher à nouveau le dictionnaire.

Exercice 6 : correction

```
# Création d'un dictionnaire vide
contact = {}

# Remplissage avec des saisies utilisateur
contact["nom"] = input("Entrez le nom : ")
contact["telephone"] = input("Entrez le téléphone : ")
contact["email"] = input("Entrez l'email : ")

# Affichage du contact
print("\nContact enregistré :")
for cle, valeur in contact.items():
    print(f"{cle} : {valeur}")

# Bonus : modification du numéro
nouveau_tel = input("\nNouveau téléphone : ")
contact["telephone"] = nouveau_tel

# Réaffichage
print("\nContact mis à jour :")
for cle, valeur in contact.items():
    print(f"{cle} : {valeur}")
```

```
fruits = ["apple", "orange",  
# print(dir(fruits))  
# print(help(fruits))  
# print(len(fruits))  
# print("pineapple" in fruits)  
  
# fruits[0] = "pineapple"  
|  
  
# print(fruits[0])  
for fruit in fruits:  
    print(fruit)
```

les tableaux (lists)

Un tableau (ou liste) est une structure **de données ordonnée** qui peut contenir plusieurs valeurs (même ou différents types).

```
nombres = [1, 2, 3, 4, 5]  
fruits = ["pomme", "banane", "kiwi"]  
melange = [True, 42, "salut"]
```

Accéder aux éléments

```
print(fruits[0]) # "pomme"  
print(fruits[-1]) # "kiwi" (dernier élément)
```

On peut parcourir facilement un tableau avec for

```
for fruit in fruits:  
    print(fruit)
```



```
fruits = ["apple", "orange",  
# print(dir(fruits))  
# print(help(fruits))  
# print(len(fruits))  
# print("pineapple" in fruits)  
  
# fruits[0] = "pineapple"  
|  
  
# print(fruits[0])  
for fruit in fruits:  
    print(fruit)
```

les ensembles (set)

Un set est une collection **non ordonnée** d'éléments **uniques**.

On ne peut pas les modifier par contre on peut en ajouter / retirer

```
animaux = {"chat", "chien", "cheval"}
```

⚠ Caractéristiques d'un set

- Pas d'indexation (pas de set[0]) → non ordonné
- Chaque élément est unique automatiquement
- Utilisé pour les opérations ensemblistes (union, intersection, etc.)

Idéal pour éliminer les doublons

Très utile pour les comparaisons ou les vérifications d'appartenance rapide.



Exercice 6a : lists

```
/*  
    1. Crée une liste vide courses. Ajoute "lait", "pain" et  
    "œufs". Demande à l'utilisateur un autre produit à ajouter à la  
    liste. Affiche tous les éléments un par un avec une boucle for.  
*/
```

```
/*  
    2. Crée une liste de notes (ex : [14, 12, 16, 8]). Calcule la  
    somme et la moyenne manuellement (sans sum()). Affiche la moyenne  
    avec 2 chiffres après la virgule.*/
```

```
/*  
    3. Crée une liste de prénoms. Crée une nouvelle liste  
    inversée manuellement (sans .reverse() ni[::-1]). Affiche les  
    deux listes.  
*/
```


Exercice 6a : corrections

✓ Exercice 1 — Liste de courses

```
python

# Création de la liste vide
courses = []

# Ajout de produits
courses.append("lait")
courses.append("pain")
courses.append("œufs")

# Demander à l'utilisateur un produit
produit = input("Entrez un produit à ajouter : ")
courses.append(produit)

# Afficher tous les éléments
print("\nListe de courses :")
for item in courses:
    print(f"- {item}")
```

💡 Points clés à retenir :

- `append()` ajoute à la fin d'une liste.
- Boucle `for` simple pour parcourir la liste.

✓ Exercice 2 — Moyenne de notes

```
python

# Liste de notes
notes = [14, 12, 16, 8]

# Calcul manuel de la somme
somme = 0
for note in notes:
    somme += note

# Calcul de la moyenne
moyenne = somme / len(notes)

# Affichage formaté à 2 décimales
print(f"Moyenne : {moyenne:.2f}")
```

💡 Points clés à retenir :

- `len(notes)` → nombre d'éléments.
- `{moyenne:.2f}` → formatage avec 2 décimales.

✓ Exercice 3 — Inverser une liste

```
python

# Liste initiale
prenoms = ["Alice", "Bob", "Charlie", "David"]

# Nouvelle liste inversée
inversee = []
for i in range(len(prenoms) - 1, -1, -1):
    inversee.append(prenoms[i])

# Affichage
print("Liste originale :", prenoms)
print("Liste inversée :", inversee)
```

💡 Points clés à retenir :

- `range(len(prenoms) - 1, -1, -1)` → parcours à l'envers.
- On construit une **nouvelle liste** (pas de modification en place).

Exercice 6b : sets

```
/*
```

```
    1. Demande à l'utilisateur de saisir plusieurs prénoms,  
    séparés par des virgules.Exemple : "Alice,Bob,Alice,Tom,Bob«
```

```
    Transforme la chaîne en liste avec .split(',').Convertis la  
    liste en set pour éliminer les doublons.Affiche le set obtenu.
```

```
/*
```

```
    2. Crée deux sets :  
    groupe_a = {"Alice", "Bob", "Claire"}  
    groupe_b = {"Claire", "David", "Emma"}  
    Affiche : Les personnes dans les deux groupes. Les personnes  
    uniques à chaque groupe. La liste totale sans doublons.
```

```
/*
```

```
    3. Crée une liste de mots interdits : ["spam", "arnaque",  
    "escroquerie"]. Demande à l'utilisateur de saisir un message. Si  
    un mot interdit est trouvé dans le message, affiche "Message  
    rejeté", sinon "Message accepté".💡 Astuce : utiliser une boucle  
    + in.*/*
```

Exercice 6b : sets

python

```
# Saisie de plusieurs prénoms séparés par des virgules
saisie = input("Entrez plusieurs prénoms séparés par des virgules : ")

# Conversion en liste
liste_prenoms = saisie.split(",")

# Nettoyage : supprimer les espaces autour des prénoms
liste_prenoms = [prenom.strip() for prenom in liste_prenoms]

# Conversion en set pour éliminer les doublons
ensemble_prenoms = set(liste_prenoms)

# Affichage
print("Prénoms uniques :", ensemble_prenoms)
```

💡 Points clés à retenir :

- `.split(",")` → découpe en liste.
- `.strip()` → supprime espaces en début/fin.
- `set(liste)` → supprime automatiquement les doublons.

✅ Exercice 5 — Comparaison de groupes

python

```
groupe_a = {"Alice", "Bob", "Claire"}
groupe_b = {"Claire", "David", "Emma"}

# Intersection (dans les deux groupes)
commun = groupe_a & groupe_b
print("Dans les deux groupes :", commun)

# Différence (uniques à A)
uniques_a = groupe_a - groupe_b
print("Uniquement dans le groupe A :", uniques_a)

# Différence (uniques à B)
uniques_b = groupe_b - groupe_a
print("Uniquement dans le groupe B :", uniques_b)

# Union (tous sans doublons)
tous = groupe_a | groupe_b
print("Tous les participants :", tous)
```

✅ Exercice 6 — Présence d'un mot interdit

python

```
mots_interdits = ["spam", "arnaque", "escroquerie"]

# Demander le message
message = input("Entrez votre message : ").lower() # minuscule pour simplifier la vérif

# Vérification
interdit = False
for mot in mots_interdits:
    if mot in message:
        interdit = True
        break # on peut arrêter dès qu'on trouve un mot interdit

# Résultat
if interdit:
    print("Message rejeté 🚫")
else:
    print("Message accepté ✅")
```

les iterations

On peut parcourir ces différents objets :

Les ensembles (set)

Non ordonnés
Pas de doublons
Pas d'indexation

```
animaux = {"chat", "chien", "chat"}  
for animal in animaux:  
    print(animal)
```

Les listes (list)

Ordonnées
Autorisent les doublons
Indexées

```
fruits = ["pomme", "banane", "kiwi"]  
for fruit in fruits:  
    print(fruit)
```

Les dictionnaires (dict)

Stockent des paires clé → valeur
Pas de doublons de clés

→ Différentes façons de parcourir : `dict.keys()` → uniquement les clés

```
utilisateur = {"nom": "Alice", "age": 25}  
  
for cle in utilisateur.keys():  
    print(cle)
```

`dict.values()` → uniquement les valeurs

```
for valeur in utilisateur.values():  
    print(valeur)  
# Alice  
# 25
```

`dict.items()` → couples (clé, valeur)

```
for cle, valeur in utilisateur.items():  
    print(cle, ":", valeur)  
# nom : Alice  
# age : 25
```



Exercice 6c : iterations

```
/*
```

```
    Crée une liste eleves avec quelques prénoms, dont certains  
    en doublon.
```

```
    Crée cette liste en set pour supprimer les doublons, puis  
    affiche le set.
```

```
    Crée un dictionnaire notes où chaque élève a une note (clé =  
    prénom, valeur = note).
```

```
    Affiche :
```

```
    Toutes les clés (noms des élèves).
```

```
    Toutes les valeurs (notes).
```

```
    Toutes les paires clé/valeur avec .items().
```

Exercice 6c : iterations

```
# 1. Création d'une liste avec doublons
eleves = ["Alice", "Bob", "Alice", "Claire"]

print("Liste originale :", eleves)

# 2. Transformation en set pour supprimer les doublons
eleves_set = set(eleves)
print("Set sans doublons :", eleves_set)

# 3. Création d'un dictionnaire avec les notes
notes = {
    "Alice": 15,
    "Bob": 12,
    "Claire": 18
}

# 4. Affichage des clés, valeurs et paires

print("\nClés (élèves) :")
for cle in notes.keys():
    print(cle)

print("\nValeurs (notes) :")
for valeur in notes.values():
    print(valeur)

print("\nPaires (clé, valeur) :")
for cle, valeur in notes.items():
    print(f"{cle} : {valeur}")
```

function = A block of reusable code that can be placed anywhere in a program and executed multiple times.

```
def happy_birthday(name, age):  
    print(f"Happy birthday to {name}")  
    print("You are old!")  
    print("Happy birthday to you")  
    print()
```

```
happy_birthday("Bro", 20)  
happy_birthday("Steve", 30)  
happy_birthday("Joe", 40)  
happy_birthday()
```

les fonctions

Bloc de code réutilisable qui réalise une tâche précise. Elle permet de structurer le programme et d'éviter les répétitions.

Pour la **définir** :

```
def dire_bonjour():  
    print("Bonjour !")
```

Puis pour l'**appeler** :

```
dire_bonjour()
```

Avec paramètre(s)

```
def saluer(nom):  
    print(f"Bonjour {nom} !")  
  
saluer("Alice")
```

Avec return

```
def addition(a, b):  
    return a + b  
  
resultat = addition(5, 3)  
print(resultat) # 8
```



Exercice 7 fonctions

/*

Écrire un programme qui utilise plusieurs fonctions pour gérer des notes d'élèves et afficher la moyenne.

Crée une fonction saisir_notes() qui demande à l'utilisateur d'entrer plusieurs notes (saisie terminée quand l'utilisateur tape -1). La fonction renvoie la liste des notes.

Crée une fonction moyenne(notes) qui reçoit une liste de notes et renvoie la moyenne.

Crée une fonction afficher_resultat(moyenne) qui affiche un message formaté : "Très bien" si la moyenne ≥ 16 "Bien" si la moyenne ≥ 14 "Assez bien" si la moyenne ≥ 12 "Passable" si la moyenne ≥ 10 "Insuffisant" sinon

Dans le programme principal : Appeler saisir_notes() Calculer la moyenne avec moyenne() Afficher le résultat avec afficher_resultat()

Exercice 7 correction

```
# Fonction pour saisir les notes
def saisir_notes():
    notes = []
    while True:
        note = float(input("Entrez une note (ou -1 pour finir) : "))
        if note == -1: # condition d'arrêt
            break
        notes.append(note)
    return notes
```

```
# Fonction pour calculer la moyenne
def moyenne(notes):
    if len(notes) == 0: # éviter la division par zéro
        return 0
    return sum(notes) / len(notes)
```

```
# Fonction pour afficher le résultat avec mention
def afficher_resultat(moyenne):
    print(f"Moyenne : {moyenne:.2f}")
    if moyenne >= 16:
        print("Très bien")
    elif moyenne >= 14:
        print("Bien")
    elif moyenne >= 12:
        print("Assez bien")
    elif moyenne >= 10:
        print("Passable")
    else:
        print("Insuffisant")
```

```
# ----- Programme principal -----
notes = saisir_notes()      # étape 1 : saisie
moy = moyenne(notes)        # étape 2 : calcul
afficher_resultat(moy)      # étape 3 : affichage
```

nommage arguments

Un keyword argument permet d'appeler une fonction en nommant les paramètres.

Cela rend l'appel plus lisible et permet de changer l'ordre des arguments.

```
1 def saluer(nom, age):
2     print(f"Bonjour {nom}, tu as {age} ans.")
3
4 # Appel classique (positional arguments)
5 saluer("Alice", 25)
6
7 # Appel avec keyword arguments
8 saluer(age=25, nom="Alice")
```

Attention : Les arguments positionnels doivent venir avant les arguments nommés (si on a les 2 cas dans un appel)



Exercice 8 arguments

/*

Écrire une fonction qui utilise des arguments nommés et des valeurs par défaut.

Consignes : Crée une fonction `creer_utilisateur(nom, age, pays="France")`.

La fonction doit afficher un message du type :
Utilisateur : Alice, 25 ans, France

Dans le programme principal, appelle la fonction :

Une fois avec arguments positionnels

Une fois avec arguments nommés dans le désordre.

Une fois en utilisant la valeur par défaut pour `pays`

Exercice 8 arguments

```
# Définition de la fonction avec une valeur par défaut pour 'pays'
def creer_utilisateur(nom, age, pays="France"):
    print(f"Utilisateur : {nom}, {age} ans, {pays}")

# ----- Programme principal -----

# 1) Appel avec arguments positionnels
creer_utilisateur("Alice", 25, "Canada")
# → Utilisateur : Alice, 25 ans, Canada

# 2) Appel avec arguments nommés dans le désordre
creer_utilisateur(age=30, nom="Bob", pays="Suisse")
# → Utilisateur : Bob, 30 ans, Suisse

# 3) Appel en utilisant la valeur par défaut pour 'pays'
creer_utilisateur("Charlie", 22)
# → Utilisateur : Charlie, 22 ans, France
```



```
def add(*args):  
    total = 0  
    for arg in args:  
        total += arg  
    return total
```

```
print(add(1, 2, 3, 4,))
```

arguments multiples

***args** permet de passer un nombre variable d'arguments ² à une fonction. Les arguments sont regroupés dans un **tuple**.

```
def ma_fonction(*args):  
    print(args) # args est de type tuple
```

***args** est utilisé pour **gérer plusieurs arguments optionnels**.
Toujours placé après les paramètres classiques.
Donne de la flexibilité aux fonctions.

Tuple= méthodes associées

<https://docs.python.org/fr/3.13/tutorial/datastructures.html>



arguments nommées multiples

```
def add(*args):  
    total = 0  
    for arg in args:  
        total += arg  
    return total
```

```
print(add(1, 2, 3, 4,))
```

****kwargs** permet de passer un **nombre variable d'arguments nommés** à une fonction.

Les arguments sont regroupés dans un **dictionnaire** (clé: valeur).

```
1 def ma_fonction(**kwargs):  
2     print(kwargs) # kwargs est un dictionnaire
```

****kwargs** = arguments **nommés variables**.

Stockés dans un **dictionnaire**.

Très utile pour des fonctions avec **options flexibles**.

Dictionnaire = méthodes associées

<https://docs.python.org/fr/3.13/tutorial/datastructures.html>



Exercice 9 args & kwargs

/*

Créer une fonction `facture(*articles, **options)` qui :

- Calcule le prix total d'une série d'articles (passés en `*args`).
- Applique des options supplémentaires (passées en `**kwargs`) comme une remise ou une TVA.

1. La fonction prend : des prix d'articles via `*args` (ex: `facture(10, 20, 30)`)
2. des options nommées via `**kwargs`, comme `:remise=10` (réduction en %) `tva=20` (ajout en %)
3. La fonction affiche :
Le sous-total
Le montant de la remise appliquée (si présente)
Le montant de la TVA appliquée (si présente)
Le total final

Exer

args

```
def facture(*articles, **options):
    """
    Calcule une facture avec une liste d'articles (args)
    et des options nommées (kwargs) : remise, tva, devise.
    """

    # ----- Étape 1 : calcul du sous-total -----
    sous_total = sum(articles)
    print(f"Sous-total : {sous_total:.2f} €")

    # ----- Étape 2 : appliquer la remise si présente -----
    remise = options.get("remise", 0) # par défaut : 0
    montant_remise = sous_total * (remise / 100)
    if remise > 0:
        print(f"Remise ({remise}%) : -{montant_remise:.2f} €")

    # ----- Étape 3 : appliquer la TVA si présente -----
    tva = options.get("tva", 0) # par défaut : 0
    montant_tva = (sous_total - montant_remise) * (tva / 100)
    if tva > 0:
        print(f"TVA ({tva}%) : +{montant_tva:.2f} €")

    # ----- Étape 4 : total final -----
    total = sous_total - montant_remise + montant_tva
    print(f"Total final : {total:.2f} €")

# ----- Programme principal -----
facture(10, 20, 30, remise=10, tva=20)
```


opérateur **in** / **not in**

```
word = "APPLE"

letter = input("Guess a letter in t

💡
if letter in word:
    print(f"There is a {letter}")
else:
    print(f"{letter} was not found")
```

.n word

ca \Local\Programs\Python\Python311

the secret word: Z

with exit code: 0

Les opérateurs **in** et **not in** permettent **de tester la présence d'un élément dans un itérable** (liste, set, dict, chaîne...).

```
fruits = ["pomme", "banane", "kiwi"]

print("pomme" in fruits)      # True
print("orange" not in fruits) # True
```

in = appartenance

not in = non-appartenance

Fonctionne avec : listes, sets, tuples, chaînes, dictionnaires (par clés)



```
# Créer une liste des carrés de 0 à 4 :
```

```
carrés = [x**2 for x in range(5)]  
print(carrés) # [0, 1, 4, 9, 16]
```

```
# Avec condition :
```

```
# Créer une liste des nombres pairs de 0 à 9 :
```

```
pairs = [x for x in range(10) if x % 2 == 0]  
print(pairs) # [0, 2, 4, 6, 8]
```

```
# 2 boucles imbriquées
```

```
pairs_produits = [x*y for x in range(3) for y in range(4)]  
print(pairs_produits) # [0, 0, 0, 0, 0, 1, 2, 3, 0, 2, 4, 6]
```

Constructeur de **list**

Une list comprehension permet de **créer une liste de manière concise et lisible à partir d'une autre liste ou d'un itérable**.

Elle remplace souvent une boucle for traditionnelle.



```
[nouvel_element for element in iterable if condition]
```

nouvel_element : ce que vous voulez mettre dans la nouvelle liste

element : chaque élément de l'itérable d'origine

if condition : optionnel, filtre les éléments selon une condition

Plus compact et lisible que les boucles for classiques

Permet de filtrer facilement avec if

Peut inclure des transformations ou calculs



Exercice 10 creation list

/*

Créez une liste nombres contenant les entiers de 1 à 10.

À partir de cette liste, créez :

- a) Une nouvelle liste carrés contenant les carrés de ces nombres.
- b) Une nouvelle liste pairs ne contenant que les nombres pairs.
- c) Une liste pairs_carrés contenant les carrés uniquement des nombres pairs.
- d) Bonus : créez une liste table_multiplication contenant tous les produits $x*y$ pour x et y allant de 1 à 3.

Exercice 10 creation list

```
# Créer une liste de nombres de 1 à 10
nombres = [x for x in range(1, 11)]
print(nombres) # [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Créer une liste des carrés des nombres
carrés = [x**2 for x in nombres]
print(carrés) # [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

# Créer une liste des nombres pairs
pairs = [x for x in nombres if x % 2 == 0]
print(pairs) # [2, 4, 6, 8, 10]

# Créer une liste des carrés uniquement des nombres pairs
pairs_carrés = [x**2 for x in nombres if x % 2 == 0]
print(pairs_carrés) # [4, 16, 36, 64, 100]

# Bonus : créer une liste avec tous les produits x*y pour x et y de 1 à 4
table_multiplication = [x*y for x in range(1, 4) for y in range(1, 4)]
print(table_multiplication) # [1, 2, 3, 2, 4, 6, 3, 6, 9]
```



```
def day_of_week(day):  
    if day == 1:  
        return "It is Sunday"  
    elif day == 2:  
        return "It is Monday"  
    elif day == 3:  
        return "It is Tuesday"  
    elif day == 4:  
        return "It is Wednesday"  
    elif day == 5:  
        return "It is Thursday"  
    elif day == 6:  
        return "It is Friday"  
    elif day == 7:  
        return "It is Saturday"  
    else:  
        return "Not a valid day"
```

Match...case (depuis v3.10)

Permet de **comparer une variable à plusieurs cas possibles**

Similaire au switch qu'on trouve dans d'autres langages

Plus puissant : supporte aussi la **déstructuration** et les **patterns**

```
match variable:  
    case valeur1:  
        # instructions si variable == valeur1  
    case valeur2:  
        # instructions si variable == valeur2  
    case _:  
        # cas par défaut (équivalent de "default")
```

Lisibilité accrue par rapport aux if/elif/else

Supporte les **motifs complexes** (tuples, listes, classes, etc.)

Le **_** permet de gérer le **cas par défaut**



Exercice 11 match case et lists

```
/*
```

```
Écris un programme qui analyse une liste de nombres et affiche  
un message différent selon son contenu :
```

```
Si la liste est vide → "La liste est vide«
```

```
Si la liste contient un seul élément → "Un seul élément : X«
```

```
Si la liste contient deux éléments → "Deux éléments : X et Y«
```

```
Si la liste contient au moins trois éléments → "Liste plus  
longue, premier élément : X"
```

Exercice 11 match case et lists

```
ma_liste = [1, 2, 3] # tu peux modifier pour tester différents cas

match ma_liste:
    case []:
        print("La liste est vide")
    case [x]:
        print(f"Un seul élément : {x}")
    case [x, y]:
        print(f"Deux éléments : {x} et {y}")
    case [x, *reste]:
        print(f"Liste plus longue, premier élément : {x}")
```

```

# -----
# mon_programme.py
# -----

# 1 Définition des fonctions
def addition(a, b):
    return a + b

def soustraction(a, b):
    return a - b

def multiplication(a, b):
    return a * b

def division(a, b):
    if b != 0:
        return a / b
    else:
        return "Erreur : division par zéro"

# 2 Code principal (point d'entrée)
if __name__ == "__main__":
    print("Programme principal lancé !\n")

    x, y = 10, 5
    print(f"{x} + {y} = {addition(x, y)}")
    print(f"{x} - {y} = {soustraction(x, y)}")
    print(f"{x} * {y} = {multiplication(x, y)}")
    print(f"{x} / {y} = {division(x, y)}")

```

Modules & main

Qu'est-ce que `__name__` ?

En Python, chaque fichier est un **module**.

La variable spéciale `__name__` contient :

- `"__main__"` si le fichier est exécuté directement.
- Le nom du module si le fichier est importé ailleurs.

Pourquoi utiliser `if __name__ == "__main__":` ?

Permet de définir un point d'entrée du programme.

Le code dans ce bloc ne s'exécute **que si le fichier est lancé directement**, pas lorsqu'il est importé.

Bonne pratique pour séparer :

- les fonctions / classes réutilisables
- le code d'exécution principal

```

def dire_bonjour():
    print("Bonjour !")

# S'exécute seulement si ce fichier est lancé directement
if __name__ == "__main__":
    dire_bonjour()

```

Si on lance le fichier → affiche Bonjour !

Si on importe ce fichier dans un autre module → rien ne s'affiche



Exercice 12 modules et main

```
/*
```

Créer un mini-projet avec deux fichiers :

utils.py

Contiendra des fonctions utilitaires :

carres_pairs(n) → renvoie la liste des carrés des nombres pairs de 0 à n. (Utiliser une compréhension de liste) analyse_liste(liste) → utilise match...case pour analyser la liste :

- liste vide → "La liste est vide«
- liste d'un seul élément → "Un seul élément : X«
- liste de deux éléments → "Deux éléments : X et Y«
- liste plus longue → "Liste plus longue, premier élément : X«

main.py

Importera les fonctions de utils.py

Dans le bloc if `__name__ == "__main__":`, il devra :

- Demander à l'utilisateur un nombre n
- Calculer et afficher les carrés pairs jusqu'à n
- Analyser la liste obtenue avec analyse_liste

Exercice 12 modules et main

```
# utils.py

def carres_pairs(n):
    """Renvoie les carrés des nombres pairs de 0 à n"""
    return [x**2 for x in range(n+1) if x % 2 == 0]

def analyse_liste(liste):
    """Analyse une liste avec match...case"""
    match liste:
        case []:
            return "La liste est vide"
        case [x]:
            return f"Un seul élément : {x}"
        case [x, y]:
            return f"Deux éléments : {x} et {y}"
        case [x, *reste]:
            return f"Liste plus longue, premier élément : {x}"
```

```
# main.py
from utils import carres_pairs, analyse_liste

if __name__ == "__main__":
    # Demander un nombre à l'utilisateur
    n = int(input("Entrez un nombre : "))

    # Générer les carrés pairs
    resultat = carres_pairs(n)
    print("Carrés pairs :", resultat)

    # Analyser la liste obtenue
    analyse = analyse_liste(resultat)
    print("Analyse :", analyse)
```

Python

Exercices récapitulatifs
(voir doc)



Python

POUR ALLER PLUS LOIN





FILE HANDLING IN PYTHON

Python

MANIPULATION DE FICHIERS



```
from pathlib import Path
```

```
chemin1 = Path("exemple.txt")
```

```
chemin2 = Path("mon_dossier")
```

```
# Vérifier si c'est un fichier
```

```
if chemin1.is_file():
```

```
    print(f"{chemin1} est bien un fichier")
```

```
# Vérifier si c'est un dossier
```

```
if chemin2.is_dir():
```

```
    print(f"{chemin2} est bien un dossier")
```

```
# is_file() / os.path.isfile() → vrai si c'est un fichier
```

```
# is_dir() / os.path.isdir() → vrai si c'est un dossier
```

Fichier détection

Pourquoi vérifier l'existence d'un fichier ?

- Éviter les erreurs lors de la lecture/écriture.
- S'assurer qu'une ressource est disponible.
- Contrôler le flux du programme.

Méthode avec `pathlib` (plus moderne que `os.path`)

```
1 from pathlib import Path
2
3 fichier = Path("exemple.txt")
4 if fichier.exists():
5     print("✅ Le fichier existe")
6 else:
7     print("❌ Le fichier n'existe pas")
```

Permet aussi de vérifier **répertoire/fichier** (`is_file()`, `is_dir()`).

: toujours vérifier avant d'ouvrir un fichier pour éviter les **Bonne pratique** plantages.



```
from pathlib import Path
```

```
chemin1 = Path("exemple.txt")
```

```
chemin2 = Path("mon_dossier")
```

```
# Vérifier si c'est un fichier
```

```
if chemin1.is_file():
```

```
    print(f"{chemin1} est bien un fichier")
```

```
# Vérifier si c'est un dossier
```

```
if chemin2.is_dir():
```

```
    print(f"{chemin2} est bien un dossier")
```

```
# is_file() / os.path.isfile() → vrai si c'est un fichier
```

```
# is_dir() / os.path.isdir() → vrai si c'est un dossier
```

Fichier ouvrir/fermer

Pourquoi fermer un fichier?

- fuite de ressources
- blocage du fichier
- erreurs possibles.

Solutions :

Possibilité d'utiliser `close` mais peu lisible...

Utiliser **with** comme un bloc indenté

```
with open("exemple.txt", "r", encoding="utf-8") as f:
    #code à exécuter sur le fichier
    #code à exécuter sur le fichier
```

Code plus lisible

Pas besoin de `close()`.

Sécurité (toujours fermé proprement).



```
from pathlib import Path
```

```
chemin1 = Path("exemple.txt")
```

```
chemin2 = Path("mon_dossier")
```

```
# Vérifier si c'est un fichier
```

```
if chemin1.is_file():
```

```
    print(f"{chemin1} est bien un fichier")
```

```
# Vérifier si c'est un dossier
```

```
if chemin2.is_dir():
```

```
    print(f"{chemin2} est bien un dossier")
```

```
# is_file() / os.path.isfile() → vrai si c'est un fichier
```

```
# is_dir() / os.path.isdir() → vrai si c'est un dossier
```

Fichier consultation

Pourquoi lire un fichier ?

- Charger des données externes (texte, logs, config).
- Traiter ligne par ligne (par ex. un CSV simple).
- Éviter de tout stocker en mémoire.

Lire tout le contenu

```
# le r pour lecture seul, le with ferme automatiquement le fichier
with open("exemple.txt", "r", encoding="utf-8") as f:
    contenu = f.read()
    print(contenu)
```

Lecture ligne par ligne

```
with open("exemple.txt", "r", encoding="utf-8") as f:
    for ligne in f:
        print(ligne.strip()) # .strip() retire \n
```

Lire en liste de lignes

```
with open("exemple.txt", "r", encoding="utf-8") as f:
    lignes = f.readlines()
    print(lignes)
```



Exercice 13 : lecture fichier

```
/*
```

```
Crée un fichier texte notes.txt contenant 5 lignes (par exemple des prénoms + des notes). Ex Bob : 15
```

```
Écris un programme qui :
```

```
Vérifie si le fichier notes.txt existe.
```

- Si non, affiche : "Le fichier notes.txt est introuvable."
- Si oui, continue.

```
Lis le contenu du fichier de 3 manières différentes :
```

- Avec read() → affiche tout le contenu en une seule chaîne.
- Avec une boucle for ligne in f → affiche chaque ligne séparément.
- Avec readlines() → affiche la liste des lignes (et par exemple la note du 3e étudiant en utilisant un index).

Exercice 12 : lecture fichier

```
from pathlib import Path

# 1. Vérification de l'existence
fichier = Path("notes.txt")

if not fichier.exists():
    print("❌ Le fichier notes.txt est introuvable.")
else:
    print("✅ Fichier trouvé !\n")

# 2. Lecture avec read()
with open(fichier, "r", encoding="utf-8") as f:
    contenu = f.read()
print("=== Lecture avec read() ===")
print(contenu) # tout en un seul bloc
print()

# 3. Lecture ligne par ligne
print("=== Lecture ligne par ligne ===")
with open(fichier, "r", encoding="utf-8") as f:
    for ligne in f:
        print(ligne.strip()) # strip() enlève les retours à la ligne
print()

# 4. Lecture avec readlines()
with open(fichier, "r", encoding="utf-8") as f:
    lignes = f.readlines()
print("=== Lecture avec readlines() ===")
print(lignes) # une vraie liste Python
print(f"👉 La 3ème entrée est : {lignes[2].strip()}")
```

Fichier écriture

Différents modes d'écriture

- "w" → write (écriture)
 - Crée un nouveau fichier ou écrase le fichier existant.
 - Attention : tout le contenu précédent est perdu.

```
with open("exemple.txt", "w", encoding="utf-8") as f:  
    f.write("Première ligne\n")  
    f.write("Deuxième ligne\n")
```

- "a" → append (ajout)
 - Ajoute du texte à la fin du fichier sans effacer l'existant.

```
with open("exemple.txt", "a", encoding="utf-8") as f:  
    f.write("Nouvelle ligne ajoutée\n")
```

Toujours utiliser with open(...) → ferme automatiquement le fichier.

Ne pas oublier \n pour sauter de ligne.

Mode "w" = destructif → à utiliser avec précaution.



```
#pour re ecrire le fichier entierement  
with open("journal.txt", "w", encoding="utf-8") as f:  
    f.write("Première ligne\n")  
    f.write("Deuxième ligne\n")
```

```
#pour ajouter une ligne au fichier existant  
with open("journal.txt", "a", encoding="utf-8") as f:  
    f.write("Nouvelle ligne ajoutée\n")
```

Exercice 13 : script de log

```
/*
```

Créer un petit script de journalisation qui enregistre dans un fichier texte toutes les entrées de l'utilisateur.

- 1) Demander à l'utilisateur d'entrer une phrase (ou quit pour arrêter).
- 2) Chaque entrée doit être sauvegardée dans un fichier log.txt avec :
 - la phrase entrée
 - la date et l'heure (ex: 2025-08-24 14:32:12 - Bonjour).
- 3) Le script doit :
 - Continuer tant que l'utilisateur ne tape pas quit.
 - Ajouter les entrées à la fin du fichier (mode "a").
 - Utiliser with open(...) pour écrire.

Utiliser une boucle while.

Pour récupérer la date et l'heure :

```
from datetime import datetime
now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```


Exercice 13 : script de log

```
from datetime import datetime

# Nom du fichier de log
fichier_log = "log.txt"

# Initialisation de la variable
entree = ""

# Boucle qui continue tant que l'utilisateur ne tape pas 'quit'
while entree.lower() != "quit":
    entree = input("Entrez une phrase (ou 'quit' pour arrêter) : ")

    if entree.lower() != "quit":
        # Récupérer la date et l'heure
        now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        # Construire la ligne à écrire
        ligne = f"{now} - {entree}\n"

        # Écriture dans le fichier en mode ajout
        with open(fichier_log, "a", encoding="utf-8") as f:
            f.write(ligne)

        print("✅ Entrée enregistrée.")

print("Fin du programme.")
```



Python

INTERAGIR AVEC UNE BDD EN SQL

SQL

Python With SQL Database

requête **SELECT**

Objectif ?

- Se connecter à une base de données MySQL (installer **mysql connector**)
- Récupérer les données d'une seule table
- Afficher les résultats dans la console

mysql.connector.connect() → Connexion à la base MySQL

curseur.execute() → Exécute la requête SQL

fetchall() → Récupère tous les résultats

Boucle for → Parcours et affichage des lignes

ATTENTION :

Conseil de tester la requête en SQL avant

Toujours fermer le curseur et la connexion

```
import mysql.connector

# Connexion à la BDD
connexion = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",
    database="bibliotheque"
)

curseur = connexion.cursor()

# Requête SELECT simple
curseur.execute("SELECT id, nom, pays FROM auteurs")

# Récupérer les résultats
resultats = curseur.fetchall()

# Affichage
print("Liste des auteurs :")
for auteur in resultats:
    print(f"ID: {auteur[0]}, Nom: {auteur[1]}, Pays: {auteur[2]}")

# Fermeture
curseur.close()
connexion.close()
```



Exercice 14 : JOIN et Python

```
/*
```

```
Connecte-toi à la base bibliotheque via Python.
```

```
Écris une requête SQL pour récupérer :
```

- Le titre du livre
- L'année de publication
- Le nom de l'auteur
- Utilise un JOIN pour lier livres.id_auteur à auteurs.id.

```
Parcours les résultats et affiche-les dans la console
```

```
exemple :
```

```
Titre: Les Misérables, Année: 1862, Auteur: Victor Hugo
```

```
Titre: 1984, Année: 1949, Auteur: George Orwell
```

```
Ferme correctement le curseur et la connexion.
```

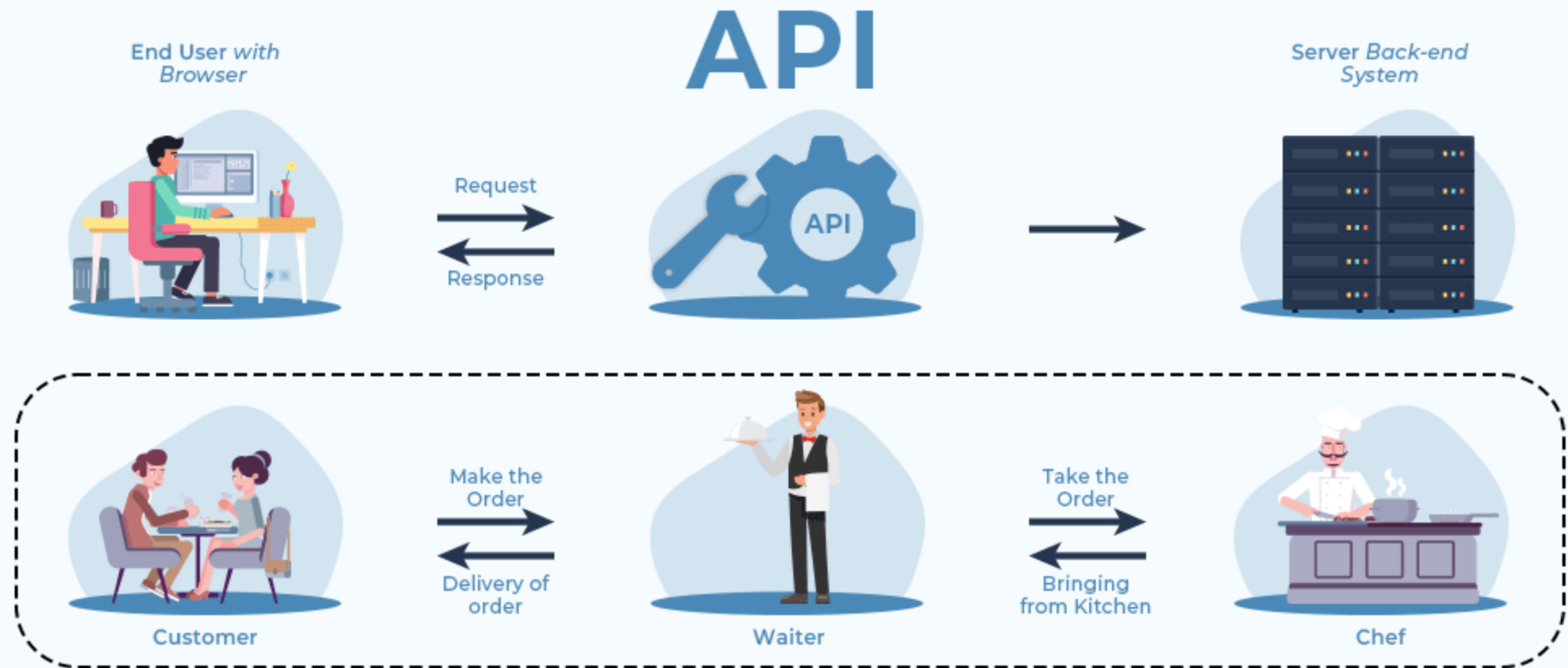



Python

REQUÊTES HTTP ET API

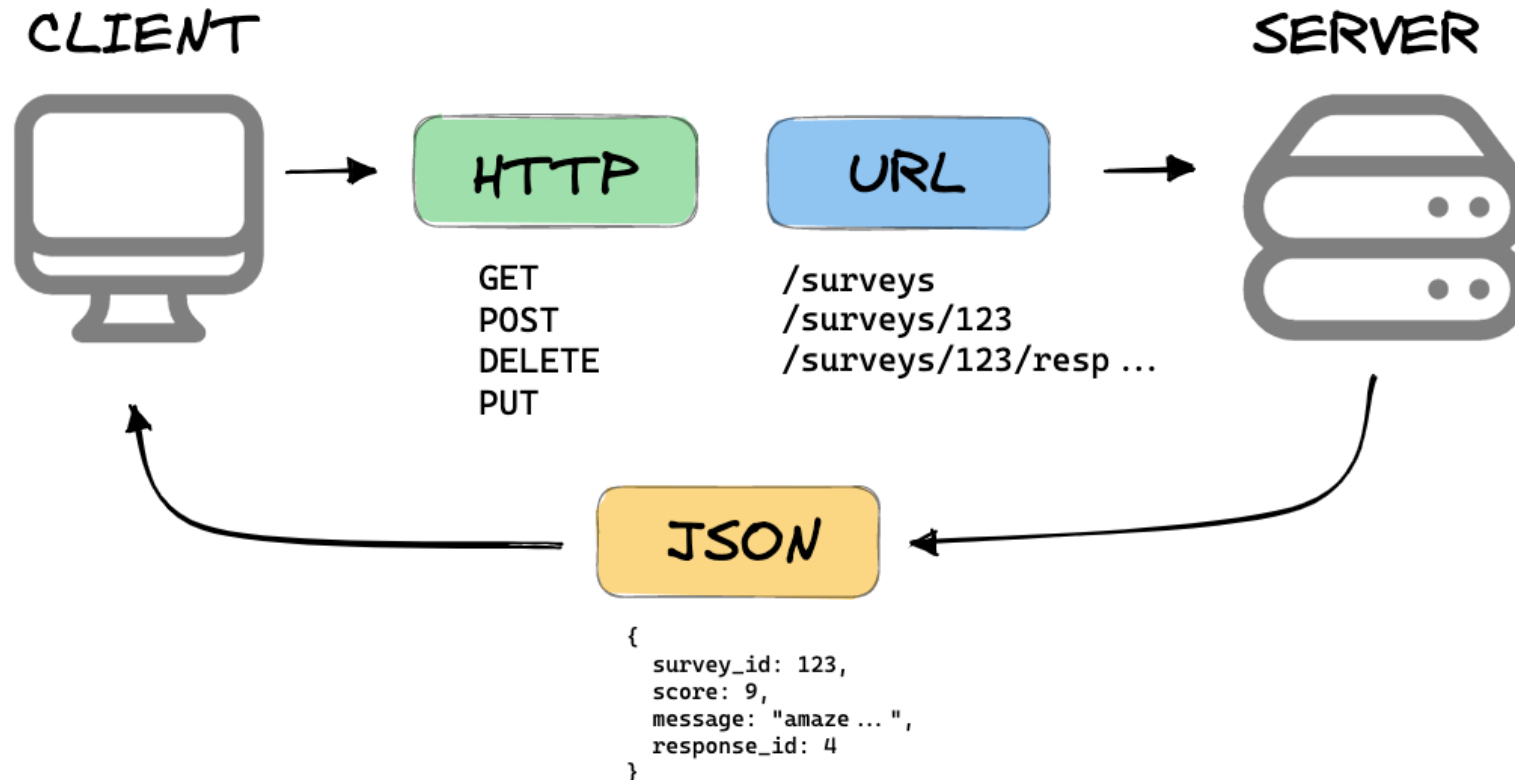


Qu'est-ce qu'une **API** ?



Source : <https://www.geeksforgeeks.org/software-testing/what-is-an-api/>

Le HTTP dans une API ?



Les méthodes HTTP courantes

GET

Récupérer des informations

POST

Envoyer des données au serveur

DELETE

Supprimer une ressource

PUT

Modifier une ressource

requête GET

Qu'est ce qu'une requête GET ?

- Méthode HTTP qui permet de demander des informations à un serveur.
- Le serveur répond avec des données (souvent JSON ou HTML).

```
# 1.import de la librairie request
import requests

# 2. renseigner URL de l'API
url = "https://api.agify.io/?name=Alice"
```

```
# 3. Envoyer notre requête GET
reponse = requests.get(url)
```

```
# 4. Vérifier le code HTTP renvoyé par le serveur
if reponse.status_code == 200:
    print("✅ Requête réussie !")
    #4b pour afficher ce que renvoi l'API
    print(reponse.text)
else:
    print(" Erreur :", reponse.status_code)
```

3. **requests.get(url)** → envoie une requête GET à l'URL.

4. **reponse.status_code** → code HTTP :

- 200 = succès
- 404 = introuvable...

5. **reponse.text** → contient le texte brut de la réponse (JSON ou HTML).

- Permet de voir ce que renvoie l'API avant de traiter le JSON.
- Très utile pour comprendre la structure des données.



Recuperer et afficher JSON

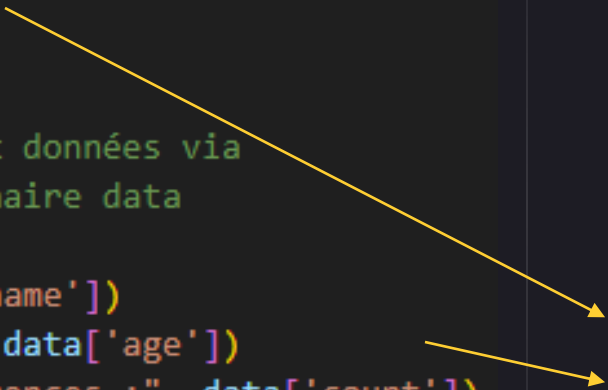
```
import requests

url = "https://api.agify.io/?name=Alice"
reponse = requests.get(url)

# Transformer le JSON en dictionnaire Python
data = reponse.json()
print(data)

# On accède ensuite aux données via
# les clés du dictionnaire data

print("Nom :", data['name'])
print("Âge estimé :", data['age'])
print("Nombre d'occurrences :", data['count'])
```



Qu'est ce que JSON ?

- JSON = JavaScript Object Notation
- Format léger pour échanger des données : clé / valeur.
- Très utilisé dans les APIs.

En **Python**, un **JSON** est souvent converti en dictionnaire

- **.json()** transforme la réponse en dictionnaire Python.
- On peut ensuite accéder aux valeurs avec les clés.



Exercice 15 : API météo

/*

Créer un petit programme qui :

Demande le nom d'une ville à l'utilisateur

Récupère la météo actuelle via une API publique

Affiche la température et le temps en clair

Lien vers l'API :

https://api.open-meteo.com/v1/forecast?latitude=48.85&longitude=2.35¤t_weather=true

Envoyer une requête GET avec requests.

Vérifier que le code HTTP est 200.

Récupérer les données JSON avec .json().

Afficher clairement :

La température actuelle

L'état du temps (ensoleillé, nuageux...)

TIPS : demander à l'utilisateur de saisir une ville et stocker dans une variable ville.

Les données sont dans un dictionnaire Python après .json().

Utiliser print() pour afficher les informations de façon lisible.

Exercice 14 : API météo

```
import requests

# Exemple avec Paris (latitude et longitude)
latitude = 48.85
longitude = 2.35

# URL de l'API Open-Meteo
url = f"https://api.open-meteo.com/v1/forecast?latitude={latitude}&longitude={longitude}&current_weather=true"

# Envoyer la requête GET
reponse = requests.get(url)

# Vérifier le code HTTP
if reponse.status_code == 200:
    data = reponse.json() # Transformer JSON en dictionnaire Python

    # Extraire les informations de météo
    meteo = data["current_weather"]
    temperature = meteo["temperature"]
    vitesse_vent = meteo["windspeed"]

    print("Météo à Paris :")
    print(f"Température : {temperature}°C")
    print(f"Vitesse du vent : {vitesse_vent} km/h")
else:
    print(" Erreur :", reponse.status_code)
```



Python

WEB SCRAPING

API vs Scraping ?



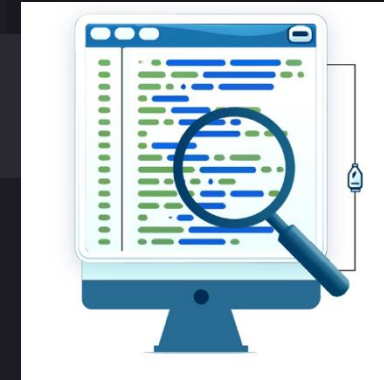
Interface officielle fournie par un service pour accéder à ses données.
Données souvent formatées **en JSON ou XML**.
Stable et **conçue** pour être utilisée par des programmes.

Avantages :

Données fiables et structurées.
Facile à utiliser avec Python (requests, json()).

Inconvénients :

Nécessite parfois une clé API.
Pas toutes les informations du site sont disponibles.



Récupération des données directement depuis le HTML d'une page web.
Utilise requests pour récupérer la page et **BeautifulSoup** pour parser le HTML.

Avantages :

Accès à toutes les données visibles sur la page.
Peut être utilisé même si aucune API n'existe.

Inconvénients :

Fragile : si le site change, le script peut casser.
Aspects légaux et éthiques à respecter (robots.txt).

Toujours préférer une API si elle existe, le scraping est un recours quand aucune API n'est disponible.

Récupérer le **HTML**

```
import requests

# 1) on stock URL de la page à récupérer
url = "https://www.example.com"

# 2) on envoi une requête GET
reponse = requests.get(url)

# 3) Vérifier que la requête a réussi
# si oui => on stock le contenu html dans une variable
if reponse.status_code == 200:
    html = reponse.text
    print("✅ HTML récupéré !")
    print(html[:500]) # Affiche les 500 premiers caractères
else:
    print("Erreur :", reponse.status_code)
```

Objectif

- Récupérer le contenu brut HTML d'une page web pour ensuite l'analyser.
- Première étape du web scraping.
- On va utiliser requests comme pour l'API

2) envoie une requête HTTP GET

3) vérifie si la requête a réussi (200 = OK).

contient le HTML complet de la page

Toujours vérifier les conditions d'utilisation du site.



Extraire contenu HTML avec BeautifulSoup

```
import requests
from bs4 import BeautifulSoup

# Récupérer le HTML
url = "https://www.example.com"
reponse = requests.get(url)
html = reponse.text

# Parser le HTML
soup = BeautifulSoup(html, "html.parser")

# Extraire tous les titres <h1>
titres = soup.find_all("h1")
for t in titres:
    print(t.text.strip())

# Extraire un élément avec une classe spécifique
paragraphe = soup.find("p", class_="intro")
if paragraphe:
    print(paragraphe.text.strip())
```

Objectif

Installer BeautifulSoup4

- Parser le HTML récupéré pour extraire uniquement les informations souhaitées.
- Identifier des balises, classes ou identifiants pour cibler les données.

BeautifulSoup(html, "html.parser") → transforme le HTML en objet

find_all("balise") → retourne tous les éléments d'un certain type.

find("balise", class_="nom_classe") → retourne le premier élément correspondant à la balise et à la classe.

text.strip() → récupère le texte propre, sans balises ni espaces inutiles.



Exercice 16 : scraping site

/*

Choisir un site d'actualité simple ou une page de test comme
:https://www.example.com/news

1) Envoyer une requête GET avec requests pour récupérer le HTML.

2) Parser le HTML avec BeautifulSoup.

3) Extraire tous les titres d'articles :

- Suppose qu'ils sont dans des balises <h2> avec éventuellement une classe title.
- Utiliser find_all("h2") ou find_all("h2", class_="title").

4)Afficher tous les titres dans la console, numérotés.

TIPS : Pour inspecter la page, utiliser clic droit → Inspecter dans Le navigateur.

.text.strip() permet de récupérer le texte propre des balises.

Boucle for pour parcourir tous les titres trouvés.

Exercice 15 : scraping site

```
import requests
from bs4 import BeautifulSoup

# URL de la page à scraper
url = "https://www.materiel.net/"

# Envoyer une requête GET
reponse = requests.get(url)

# Vérifier que la requête a réussi
if reponse.status_code == 200:
    # Parser le HTML
    soup = BeautifulSoup(reponse.text, "html.parser")

    # Extraire tous les titres d'articles
    titres = soup.find_all("h2")

    # Afficher les titres
    for i, titre in enumerate(titres, start=1):
        print(f"{i}. {titre.text.strip()}")
else:
    print("Erreur lors de la récupération de la page :", reponse.status_code)
```